Tutorial for Diffusion Model

What is Diffusion Model (DDPM)?

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Denoising diffusion probabilistic models

[PDF] neurips.cc

J Ho, A Jain, P Abbeel - Advances in neural information ..., 2020 - proceedings.neurips.cc

... This paper presents progress in diffusion probabilistic models [53]. A diffusion probabilistic model (which we will call a "diffusion model" for brevity) is a parameterized Markov chain ...

☆ 保存 99 引用 被引用次数：28666 相关文章 所有 10 个版本 »

What is Diffusion Model?

- The diffusion model is a class of generative models widely used for images, videos, and audio.

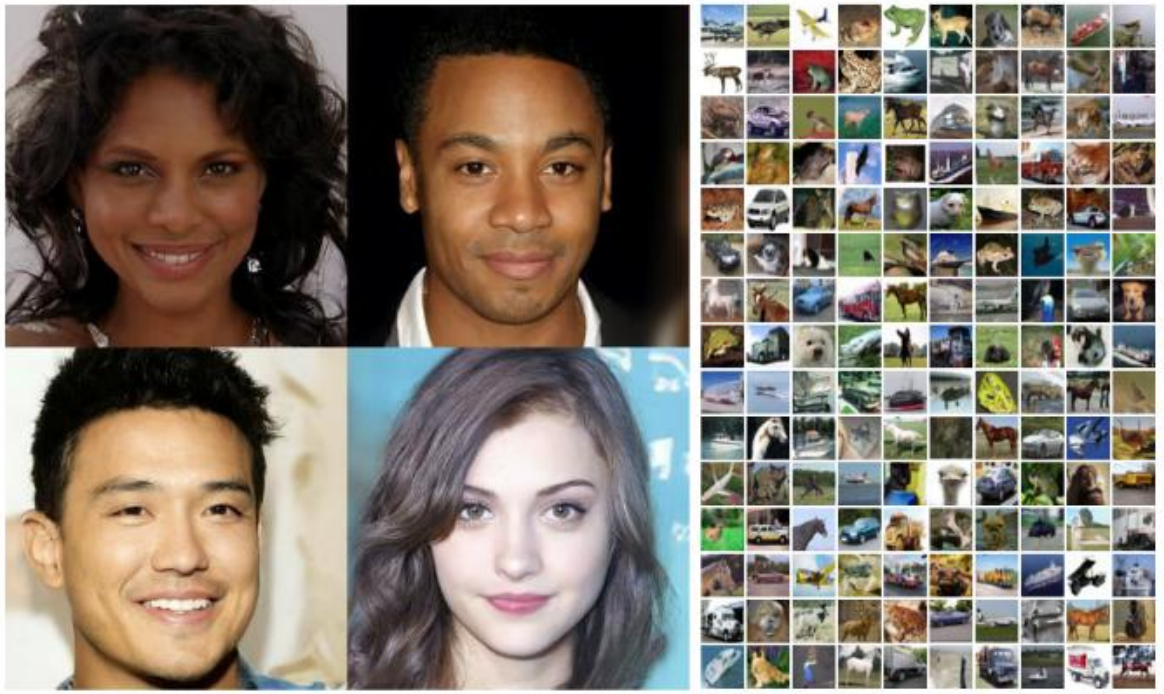


Figure 1: Generated samples on CelebA-HQ 256 × 256 (left) and unconditional CIFAR10 (right)

How does Diffusion Model work?

- Diffusion Model works by gradually adding random noise to data through a forward diffusion process, and then learning to reverse this process to reconstruct or generate realistic samples from pure noise.

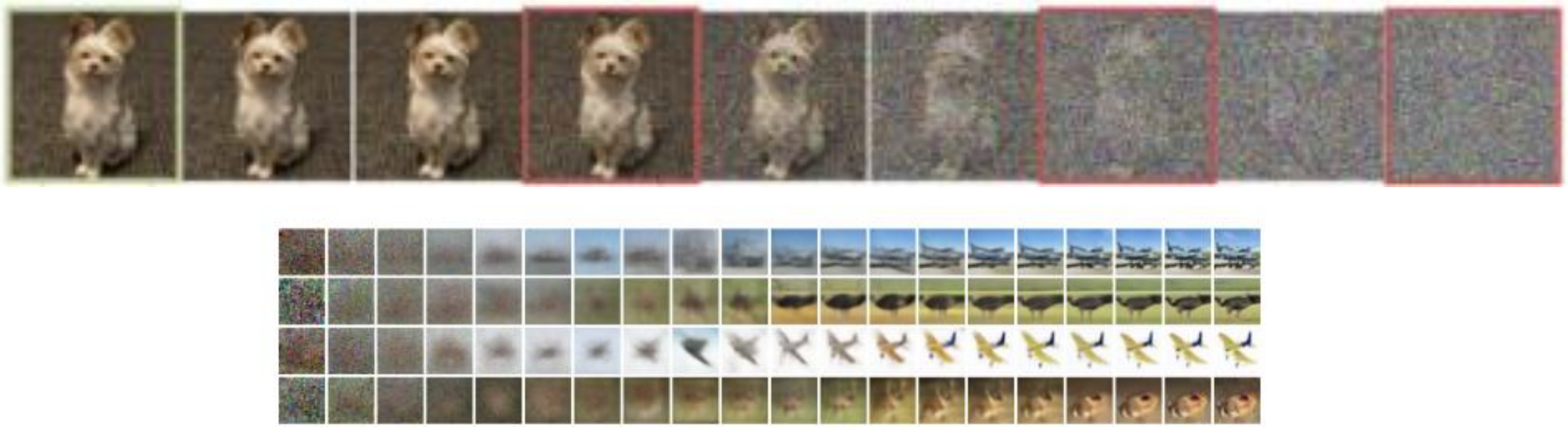


Figure 6: Unconditional CIFAR10 progressive generation ($\hat{\mathbf{x}}_0$ over time, from left to right). Extended samples and sample quality metrics over time in the appendix (Figs. 10 and 14).

How does Diffusion Model work?

- Goal: Suppose we have 100 different pictures of dogs (denoted as X_0), and we want to generate a new picture of a dog that looks realistic but different from any of the existing ones.
- Step 1: Add a little bit of noise to the original picture of dogs (denoted as X_1)
- Step 2: Add a little bit of noise to X_1 (denoted as X_2)
- Step 3: Repeat this process until we have a picture of noise
- Step 4: Train a neural network to denoise from X_t to X_{t-1} until X_0
- Step 5: Now we have a neural network that can start from pure noise and denoise it step by step to generate a new, realistic picture of a dog.

X_0 X_1 X_2 X_3 …

A taste of Math

- Stochastic Process
  - We add random noise to images. So the sequence (x_0,x_1,…,x_t) is a stochastic process, and we use $q(x_t)$ to describe the probabilistic distribution of x_t within this process.
  - In a Stochastic process, we often optimize expectations with respect to probability distributions.
- Gaussian Distribution (Also known as Normal Distribution)
  - A Gaussian (or Normal) distribution often arises as the sum of many independent random variables.
  - Since real-world noise can usually be modeled as the combination of many small random effects, it is common to assume that the added noise follows a Gaussian distribution. Thus, we use Gaussian noise in diffusion models.

A taste of Math

- Conditional Probability
  - $q(x_t)$ represents the (marginal) probability distribution of x_t.
  - When the original clean image x_0 is known, $q(x_t|x_0)$ denotes the conditional probability distribution of x_t given x_0

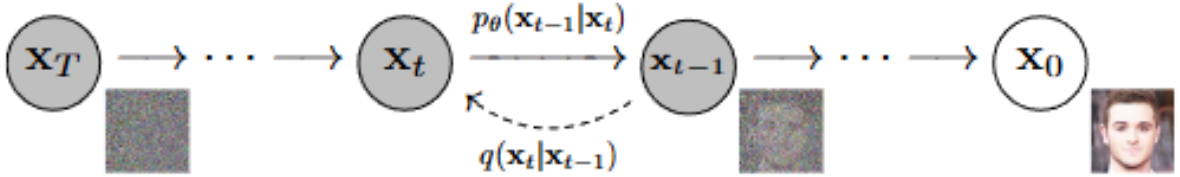
- KL Divergence
  - The Kullback–Leibler (KL) divergence measures how one probability distribution differs from another.

$$D_{\mathrm{KL}}(P \parallel Q) = \int P(x) \log \frac{P(x)}{Q(x)} \, dx$$

Key Insights Behind Diffusion Models

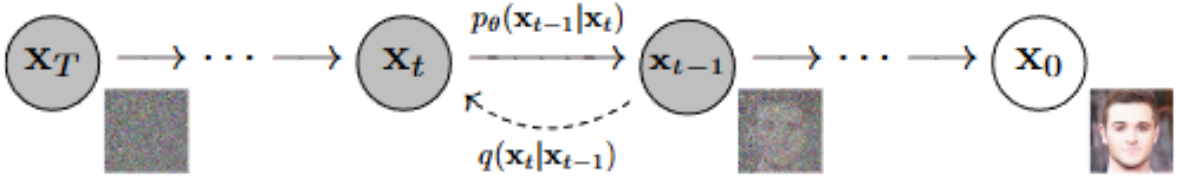
1. Markov Property
   - each noisy image x_t depends only on the previous step x_{t-1}, not on earlier steps.
   - $q(x_t|x_{t-1},x_{t-2},...,x_0)=q(x_t|x_{t-1})$

2. Forward Diffusion Process (mathematical definition of “adding noise”)
   - $x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$
   - $q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I)$
   - $q(x_{1:T}|x_0) := \prod_{t=1}^{T} q(x_t|x_{t-1})$
   - $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$ using the notation $\alpha_t := 1-\beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$



Key Insights Behind Diffusion Models

- 3. Reverse Diffusion Process ("denoising") and Variational Inference
  - We aim to train a neural network (U-net) to perform denoising, in other words, to approximate $q(x_{t-1}|x_t)$.
  - During training, the original x_0 is known, so the true posterior $q(x_{t-1}|x_t,x_0)$ can be computed analytically using the Bayesian rule. $q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1},\mathbf{x}_0)\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$
  - However, during generation, x_0 is unknown. We employ variational inference by introducing a parameterized model $p_\theta(x_{t-1}|x_t)$ to approximate the true posterior $q(x_{t-1}|x_t)$, then minimizing the KL divergence.
  - Although $q(x_{t-1}|x_t, x_0)$ is known during training, we still use $p_\theta(x_{t-1}|x_t)$ to approximate it, so that the learned model can later be used for denoising (generation) when x_0 is not available.



Key Insights Behind Diffusion Models

- 4. Loss Function
  - The loss function can be derived from the Evidence Lower Bound (ELBO) in Variational Inference.
  - The exact mathematical form is theoretical and complicated. But we can understand it intuitively and rewrite it as three intuitive objectives.
  - Diffusion loss just wants three things:
    - At the beginning, The model's prior $p(x_T)$ should match the true noisy distribution $q(x_T|x_0)$. $L_T = D_{KL}\big(q(x_T|x_0) \,\|\, p(x_T)\big)$
    - In the middle, The model's reverse process $p_\theta(x_{t-1}|x_t)$ should behave like the true diffusion process $q(x_{t-1}|x_t,x_0)$.

      $$L_t = D_{KL}\big(q(x_{t-1}|x_t, x_0) \,\|\, p_\theta(x_{t-1}|x_t)\big)$$

    - At the end, the generated sample x_0 should have no remaining noise.

      $$L_0 = \mathbb{E}_{x_0,t,\epsilon\sim\mathcal{N}(0,I)}\big[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\big]$$

    - Therefore, $L_{\text{VLB}} = L_T + L_{T-1} + \cdots + L_0$

$$
\begin{aligned}
L_{\text{CE}} &= -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0) \\
&= -\mathbb{E}_{q(\mathbf{x}_0)} \log \Big(\int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \Big) \\
&= -\mathbb{E}_{q(\mathbf{x}_0)} \log \Big(\int q(\mathbf{x}_{1:T}|\mathbf{x}_0) \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} d\mathbf{x}_{1:T} \Big) \\
&= -\mathbb{E}_{q(\mathbf{x}_0)} \log \Big(\mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \Big) \\
&\le -\mathbb{E}_{q(\mathbf{x}_{0:T})} \log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \\
&= \mathbb{E}_{q(\mathbf{x}_{0:T})} \Big[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \Big] = L_{\text{VLB}}
\end{aligned}
$$

$$
\begin{aligned}
-\log p_\theta(\mathbf{x}_0) &\le -\log p_\theta(\mathbf{x}_0) + D_{\text{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0) \| p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \qquad \text{; KL is non-negative} \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T}\sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \Big[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T}) / p_\theta(\mathbf{x}_0)} \Big] \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_q \Big[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} + \log p_\theta(\mathbf{x}_0) \Big] \\
&= \mathbb{E}_q \Big[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \Big] \\
\text{Let } L_{\text{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \Big[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \Big] \ge -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0)
\end{aligned}
$$

$$
\begin{aligned}
L_{\text{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \Big[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \Big] \\
&= \mathbb{E}_q \Big[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \Big] \\
&= \mathbb{E}_q \Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \Big] \\
&= \mathbb{E}_q \Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \Big] \\
&= \mathbb{E}_q \Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \Big(\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \cdot \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} \Big) + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \Big] \\
&= \mathbb{E}_q \Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \Big] \\
&= \mathbb{E}_q \Big[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{q(\mathbf{x}_1|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \Big] \\
&= \mathbb{E}_q \Big[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \Big] \\
&= \mathbb{E}_q [\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p_\theta(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}]
\end{aligned}
$$

Key Insights Behind Diffusion Models

- 5. Simplified Loss Function
  - Empirically, we can use a simplified loss function to replace the ELBO loss.
  - Our neural network is trained to predict (remove) the added Gaussian noise at each step. In other words, we only need to let the predicted $\varepsilon_\theta(x_t,t)$ approximate the real noise ε.
  - Therefore, the simplified and real-practical loss function is as follows.

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t,x_0,\varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t,t)\|^2\right]$$

Experiments

Figure 11: CelebA-HQ 256 × 256 generated samples

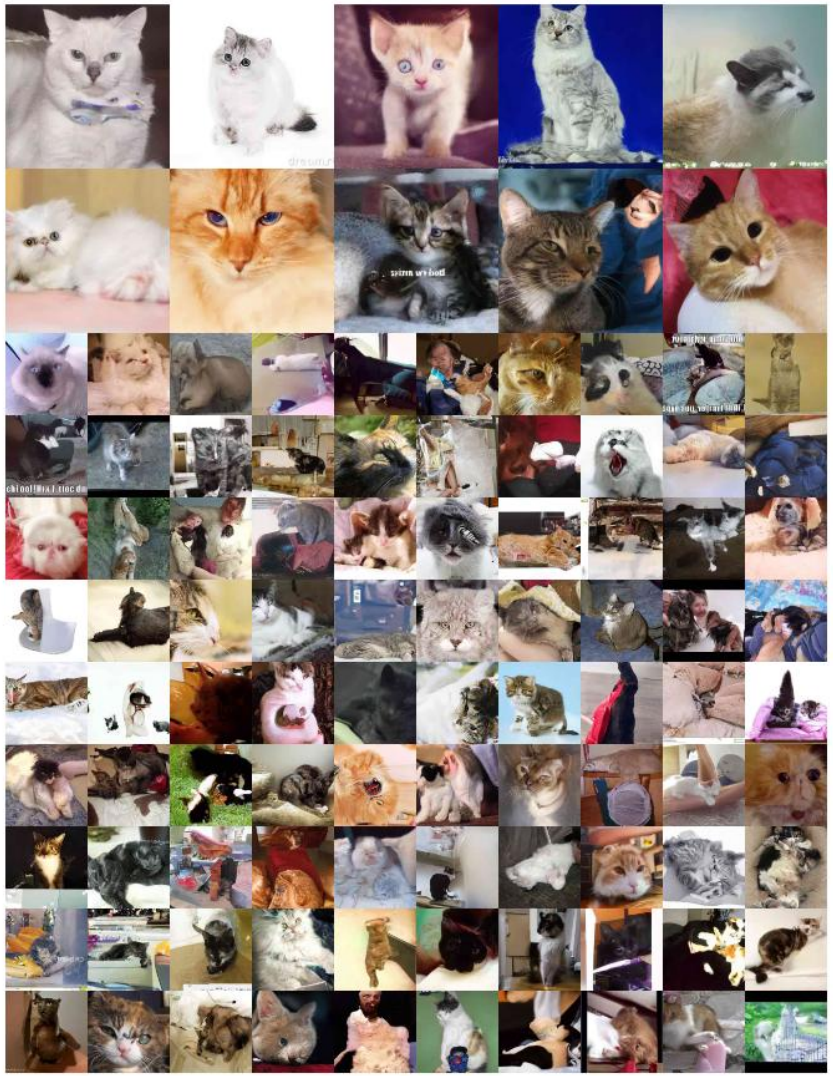


Figure 19: LSUN Cat generated samples. FID=19.75

Experiments

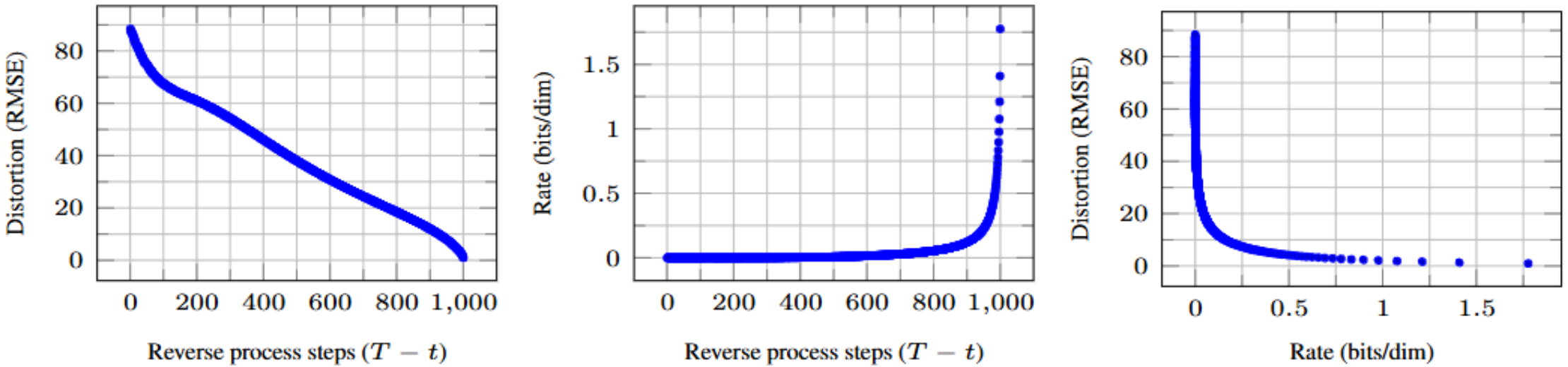


Figure 5: Unconditional CIFAR10 test set rate-distortion vs. time. Distortion is measured in root mean squared error on a $[0, 255]$ scale. See Table 4 for details.

Experiments

Figure 6: Unconditional CIFAR10 progressive generation ($\hat{\mathbf{x}}_0$ over time, from left to right). Extended samples and sample quality metrics over time in the appendix (Figs. 10 and 14).

Why do diffusion models work?

- Define the gradient of the data distribution (score function) as follows

$$s(x) = \nabla_x \log p(x),$$

- With the score, we can iteratively update a noisy sample toward the data using $x_{k+1} = x_k + \frac{\eta}{2} s(x_k) + \sqrt{\eta}\, z_k, \quad z_k \sim \mathcal{N}(0, I)$ (similar to Taylor expansion)

- Thus, diffusion models aim to learn the score function $s_\theta(x,t)$ that approximates the true gradient of the data distribution.
- It can be viewed as a probabilistic version of gradient ascent (descent)
- The original Diffusion Model is simply a discrete-time approximation of this continuous process.

Denoising Diffusion Implicit Model (DDIM)

- In a word, DDPM is “generating” at every denoising step; DDIM only generates the Gaussian noise, and the denoising steps are **fixed**.
- Assume that the variance at each denoising step is zero, and the probabilistic reverse process is determined by Expectation.
- Then, given X_t, we can compute X_0 $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$ $x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$
- So each denoising step becomes deterministic

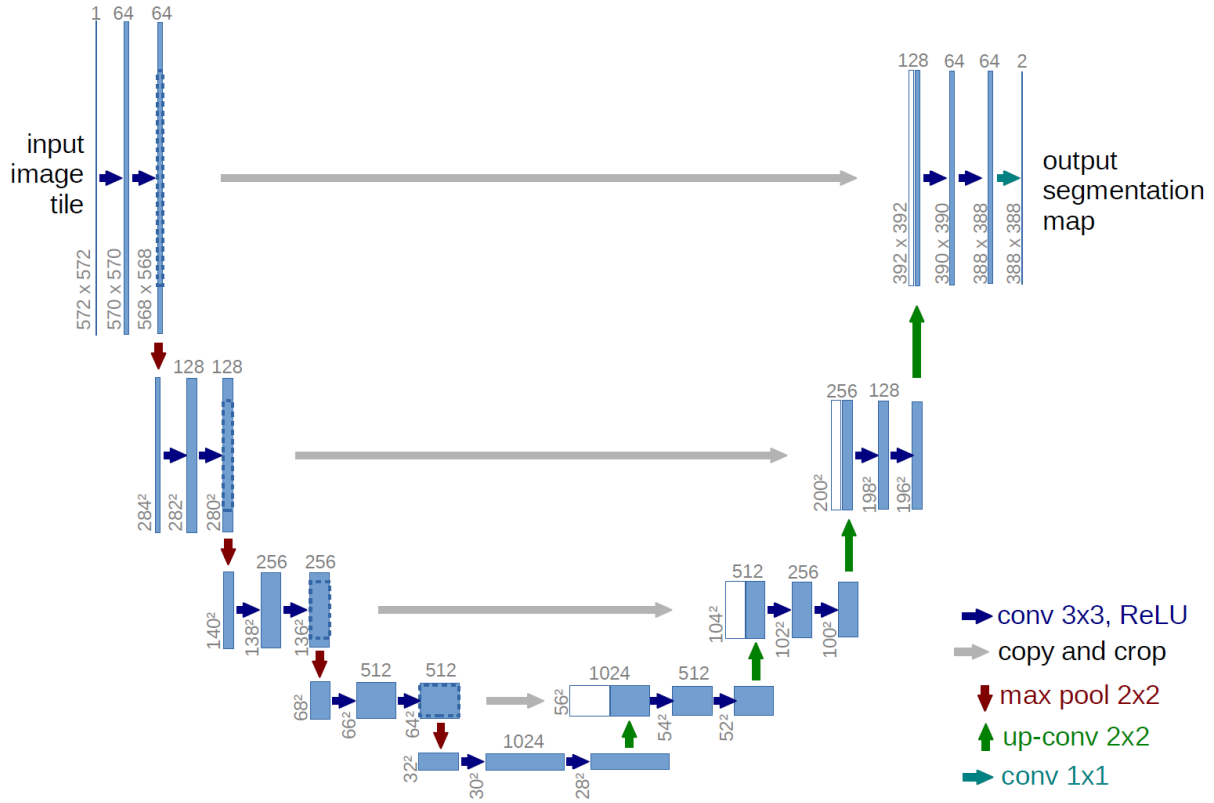
$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1-\bar{\alpha}_{t-1}}\, \epsilon_\theta(x_t, t)$$

- As a result, DDIM reduces the training step from 1000 to 50.

U-Net in Diffusion Models

- U-Net consists of a U-shaped Encoder–Decoder
  - Downsample → capture context
  - Upsample → recover detail
  - Skip-connection

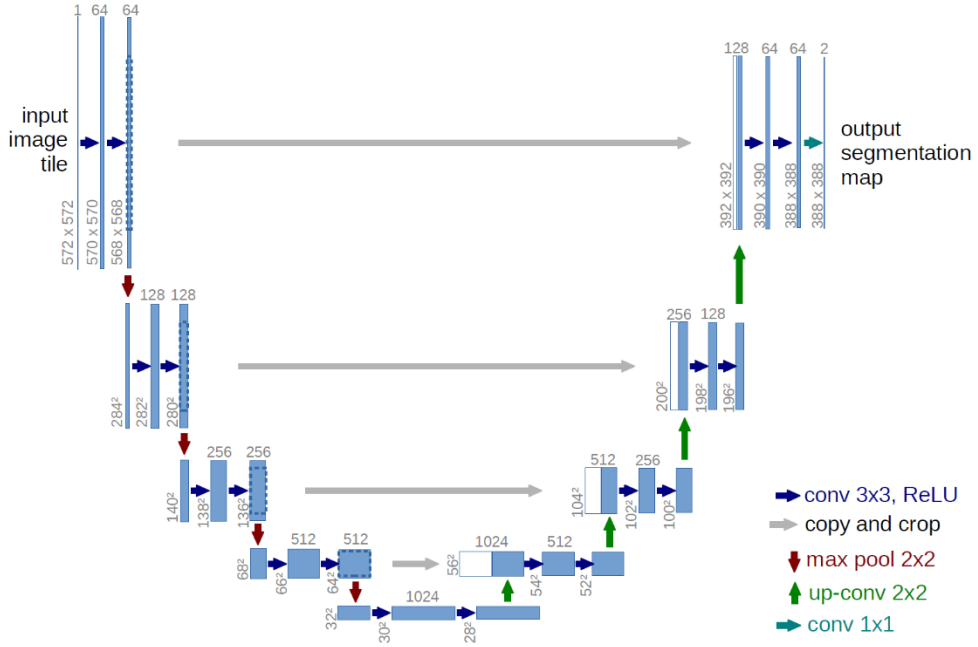
- Why U-Net works for Diffusion
  - Combines local (convolution) and global (up/downsampling) information.
  - Simple and stable architecture, well-suited for same-size image-to-image prediction tasks.



Stable Diffusion

High-resolution image synthesis with latent diffusion models [PDF] thecvf.com
R Rombach, A Blattmann, D Lorenz... - Proceedings of the ..., 2022 - openaccess.thecvf.com
... This enables efficient exploration of a large number of diffusion models for various image-to-image ...
We propose latent diffusion models (LDMs) as an effective generative model and a ...
☆ 保存 99 引用 被引用次数：24754 相关文章 所有 11 个版本 »

- Use a pretrained VAE to map images to a low-dimensional latent, perform diffusion(add noise/denoising) there, then decode back to pixels.
- The latent space (i.e., 64×64×4) is much smaller than the pixel space (i.e., 512×512×3), so the training is 10 times faster.



Transformer in Diffusion Models (DiT)

- The overall diffusion process remains the same with Stable Diffusion. Only use Transformer to predict $\hat{\epsilon}_\theta(z_t, t)$
- We first apply a Convolutional VAE encoder to map the input image into a latent space z_0.
- Then add noise to z_0, patchify the noisy latent, and project each patch into token embeddings.
- A Transformer operates on these tokens to perform denoising in the token space.
- Then reshape back to latent space and image.

Scalable diffusion models with transformers [PDF] thecvf.com

W Peebles, S Xie - Proceedings of the IEEE/CVF ..., 2023 - openaccess.thecvf.com

... We train latent diffusion models of images, replacing the commonly-used U-Net ... a transformer that operates on latent patches. We analyze the scalability of our Diffusion Transformers (...

☆ 保存 99 引用 被引用次数：3854 相关文章 所有 9 个版本 »